

RAG 面试知识体系详解

检索增强生成 · 从原理到工程落地的完整知识框架

本文档覆盖 11 大板块：

- ① 为什么需要 RAG
- ② 完整链路拆解
- ③ 切片策略 Chunking
- ④ Embedding 详解
- ⑤ 向量数据库
- ⑥ 优化思路全景
- ⑦ 长文档与代码检索
- ⑧ Agentic RAG
- ⑨ 排障诊断方法论
- ⑩ 评估体系与指标
- ⑪ 面试回答框架

依据「卡码笔记 · RAG 系列」11 篇文章知识整理 · 2026 年 9 月

目录 · 知识框架总览

本文的知识骨架可以概括为一条主线：「**为什么需要** → **怎么搭** → **怎么优化** → **怎么排障** → **怎么评估** → **怎么答**」。前两章建立动机与链路全景，第 3–5 章深入离线侧三大基础设施（切片、Embedding、向量数据库），第 6–8 章覆盖在线侧优化与演进方向（优化手段、长文档/代码场景、Agentic RAG），第 9–10 章是工程化的分水岭（排障诊断、评估体系），第 11 章收束为面试表达。

1. 为什么需要 RAG —— 大模型四大局限与 RAG 的应对
2. RAG 完整链路 —— 离线阶段与在线阶段
3. 切片策略 Chunking —— 四种方式对比与 Overlap
4. Embedding —— 语义压缩、选型与 Rerank 的区别
5. 向量数据库 —— 为什么不用 MySQL、ANN 索引原理
6. RAG 优化思路 —— Query / 检索 / 精排 / Context 四侧
7. 长文档与代码检索 —— 结构化切分、Parent-Child、版本与权限
8. Agentic RAG —— 演进四阶段与核心原理
9. RAG 排障诊断 —— 检索侧五类 + 生成侧四类问题
10. RAG 评估体系 —— Recall/Precision/MRR/NDCG 与 RAGAS
11. 面试回答框架 —— 五大落点、通用五步、高频八问
12. 重点难点提示与复习建议

一、为什么需要 RAG

1.1 从一个真实困境说起

假设公司有几千份内部文档：产品手册、操作规范、历史项目经验、客户合同.....要做一个 AI 问答系统，直接用大模型会遇到四个绕不开的问题：模型对内部文档一无所知、回答全靠"编"；文档上周刚更新但模型说的还是老版本；回答很流畅却无法判断真假、无法追溯来源；文档全塞进 Prompt 则 token 成本高到无法接受。这不是模型能力不够，而是大模型作为一项技术的根本性天花板——RAG 正是为突破这些天花板而诞生。

1.2 大模型的四大局限

局限	机制与原因	关键细节
幻觉问题 (Hallucination)	大模型本质是概率语言模型，生成每个 token 时做的是"预测下一个最可能的词"，而不是"从知识库查询正确答案"。不知道答案时它不会说"我不知道"，而是生成一个"听起来最合理"的答案——可能完全是捏造的，但语气坚定、格式工整。	企业场景中危害远比"回答错误"更严重：它会以 可信的方式传播错误信息 ，用户毫不知情。
私有知识盲区	大模型只在公开数据上训练，公司内部文档、客户合同、会议记录它从未见过。	把文档全塞 Prompt：少量可行；几百上千份时 token 成本与上下文长度成为瓶颈，且模型在极长上下文中 注意力严重稀释 ，对中间内容利用率极低。
知识更新滞后	训练有截止日期，通常落后现实 6 个月到一年。	不可能每隔几个月重训一次模型来"更新知识"，成本与时间都不可接受。
无法溯源	模型无法告诉你答案"来自哪份文档第几页"。	法律、医疗、金融等对准确性要求极高的场景，没有来源引用的答案往往不可接受。

1.3 RAG 的核心思路与四大应对

RAG（Retrieval-Augmented Generation，检索增强生成）的核心思路非常直觉：**在让模型回答之前，先去知识库检索出最相关的内容，把这些内容作为上下文一起喂给模型，让模型"看着资料"来回答。**

问题	RAG 的应对
幻觉	检索到的原文片段作为上下文传入，模型被要求"根据以下资料回答"。有事实依据约束后幻觉概率 大幅降低，但不是归零 。
私有知识	知识库完全由自己构建和维护，可包含任何内部文档；模型只需在被检索到的片段基础上推理和组织语言。

知识更新	更新知识库只需更新文档、重新向量化索引，不需要重训模型；知识新鲜度与模型训练截止日期解耦。
可追溯性	每次检索都知道命中了哪份文档哪个片段，可在答案中附上引用来源供用户核查。

1.4 适用与不适用场景

- **适合 RAG：**知识密集型问答（企业知识库、产品手册问答、法规查询——事实准确性要求高、内容量大、需要来源追溯）；内容频繁更新的业务（新闻摘要、竞品分析、政策解读——知识库持续接实时数据，模型不动）；长尾事实查询（"上海办公地址是什么""合同甲方是谁"）。
- **不适合 RAG：**创意生成、强推理任务、模型训练数据已良好覆盖的稳定领域——这些场景加 RAG 只是增加延迟和成本，没有实质收益。

面试 Q1：为什么有了大模型还需要 RAG？

大模型有几个局限性——幻觉、私有知识盲区、知识时效性、无法溯源。RAG 通过在回答前先检索外部知识库，把相关文档片段注入上下文，让模型"看资料回答"，有针对性地缓解以上问题。

面试 Q2：RAG 能解决幻觉问题吗？

能显著缓解，但无法根治。有了检索事实做锚点，乱编概率大幅降低，且答案可附来源核查。但模型仍可能在检索结果上做出错误推理，或在检索质量差时进行"补充生成"。RAG 降低幻觉概率，不消灭幻觉根因。

面试 Q3：什么场景推荐 / 不推荐 RAG？

适合：知识密集、内容频繁更新、需要溯源。不适合：创意生成、强推理、训练数据已覆盖的稳定领域。

一句话总结：RAG 不是大模型能力的升级，而是大模型的外置数据库——让模型回答时能检索自己的 cheat sheet，从而有据可依。

二、RAG 完整链路：离线与在线

整条链路分两段：离线侧负责"把知识存进去"，在线侧负责"把相关知识找出来、组合出答案"。

2.1 离线阶段（五步）

1. **文档加载 (Document Loading)**：知识来源多样——PDF 报告、Word、网页、Markdown、数据库记录、邮件，不同格式需要不同解析方式。这一步的质量直接影响系统上限：扫描件或排版混乱的 PDF 解析出的文本充满噪声，后续所有环节都会受损，"垃圾进，垃圾出"在 RAG 里体现得非常明显。
2. **文档处理 (清洗与预处理)**：去掉页眉页脚、无意义格式符号、重复内容；识别并保留标题结构；过滤表格乱码、图片占位符。看起来琐碎，但实际项目里**工程量最大、最容易被低估**。
3. **切片 (Chunking)**：清洗后的文档切成更小的片段 (chunk)。这是 RAG 里**设计决策最多**的环节。为什么要切？一篇 20 页文档用户问题可能只和某一段相关：整篇做检索单元，要么粒度太粗（命中整篇但相关内容被淹没），要么上下文太长（放不进模型或注意力稀释）。切多大没有通用答案，需按文档类型、上下文窗口、业务问题颗粒度决定。
4. **向量化 (Embedding)**：每个 chunk 经 Embedding 模型转为高维浮点向量（代表语义）。关键点：用户问题和文档 chunk **必须用同一个 Embedding 模型**，两者向量才处于同一语义空间，相似度计算才有意义。同时要存**元数据**：来自哪份文档、哪一页、创建时间等——"只看最近三个月的文档"这类过滤就依赖元数据。
5. **存入向量数据库**：向量和元数据分别存入向量库（Milvus、Weaviate、Chroma、Pinecone 等）和普通存储。向量库核心能力是**近似最近邻搜索 (ANN)**：在数百万向量中毫秒级找到与查询向量最相似的 top-K。

2.2 在线阶段（五步）

1. **Query 处理 (可选)**：Query 改写——把口语化问题转成更适合检索的形式，或把复杂问题拆成子问题分别检索（"报销流程和上限分别是什么"拆成两个检索任务）；Query 扩展——同义词扩展提高召回覆盖面。
2. **检索 (Retrieval)**：Query 向量化后与所有 chunk 向量做相似度计算（通常余弦相似度），召回 top-K (K 通常 3-10)。更完整的实现做**混合检索 (Hybrid Search)**：同时跑向量检索（语义）和关键词检索（精确匹配，如 BM25），两路合并，兼顾"语义理解"与"关键词精准匹配"。
3. **Rerank (精排)**：初步召回的 top-K 相关性不一定都高。用专门的 Cross-Encoder 模型对 (Query, Chunk) 对打分重排，只保留最相关的几条。RAG 优化里最常见也最有效的手段之一，代价是多一次模型推理延迟。
4. **上下文构建 (Context)**：最终 chunk + 元数据（来源文档、页码）按格式拼装成上下文，连同用户问题构建最终 Prompt。
5. **生成 (Generation)**：LLM 基于上下文生成回答。关键：Prompt 里要有明确的引导指令——优先依据资料回答而非依赖参数知识，并要求标注来源。

Prompt 结构示例：

"你是一个企业知识库助手。请根据以下资料回答问题，如果资料中没有相关信息，请明确说明。

参考资料：[来源：产品手册第3页] ... (chunk 内容) ... 用户问题：我们的退款政策是什么？"

2.3 链路的三个核心认知

- **每个环节都影响质量，但方式不同**：Chunking 决定"能不能检索到相关内容"，Embedding 模型决定"语义理解是否准确"，Rerank 决定"召回的 top 结果是否真的最相关"，Prompt 设计决定"模型是否能正确利用上下文"。优化 RAG 本质是**找到当前系统的薄弱环节**，而不是无差别调参。
- **离线和在线必须一致**：Embedding 模型、清洗方式，离线建索引怎么做在线就必须一样。索引用 A 模型建、检索用 B 模型，两边向量空间不同，相似度计算完全失效。
- **检索的目标是精准，不是全面**：top-3 高质量 chunk 通常优于 top-20 混杂结果。上下文越长，模型注意力越分散，信噪比越低。

2.4 常见误区

- **"RAG = 向量检索"**：向量检索只是在线侧一步。完整系统还包括文档解析、Chunking 策略、Embedding 选型、元数据管理、Rerank、Prompt 设计，缺一环都会拖累整体。
- **"模型够强，Chunking 随便切"**：Chunking 是最底层的基础设施，模型能力无法弥补检索质量缺陷。
- **"Rerank 一定要加"**：有延迟和成本代价。实时性要求高或文档量小的场景，精准 Embedding + 合理 top-K 往往已足够，先评估再决定。

面试 Q：请描述完整的 RAG 链路。

分两段答——离线侧：文档加载 → 清洗预处理 → Chunking → Embedding 向量化 → 存入向量数据库（同时存元数据）。在线侧：用户 Query →（可选 Query 改写）→ Query 向量化 → 向量检索召回 top-K →（可选混合检索、Rerank 精排）→ 拼装 Context + Prompt → LLM 生成 → 输出带来源引用的答案。

面试 Q：哪些环节最影响效果？

离线侧最关键：Chunking 策略（检索粒度）与 Embedding 选型（语义理解质量）；在线侧最关键：Prompt 设计。文档质量是前提——再好的系统也处理不好乱码和结构混乱的输入。

面试 Q：Embedding 模型离线在线需要一致吗？为什么？

必须一致。不同模型向量空间不同，索引用 A 建、查询用 B 生成，余弦相似度完全失去意义，检索结果会非常混乱。

三、切片策略 Chunking

3.1 核心矛盾：教科书卡片比喻

把 500 页教材撕成复习卡片：切太细（每张一句话）上下文不完整；切太粗（每张一整章）要看的内容太多，需要的句子藏在一堆无关文字里。RAG 的 Chunking 面对的就是这个矛盾：**chunk 太小，单个片段语义不完整；chunk 太大，检索出来的内容信噪比太低。**

3.2 四种主流切分策略对比

策略	做法	优点	缺点与适用
固定长度 Fixed-size	按字符数或 token 数固定切（如每个 512 token），切完拉倒	实现极简，不需要理解文档结构	完全不管语义边界，可能把一句话从中间切断；只适合原型快速验证， 不适合生产
递归字符 Recursive Character	按分隔符优先级递归尝试：先用 <code>\n\n</code> （段落）切，还大再用 <code>\n</code> 切，再用句号切……直到每个 chunk 在目标大小内	倾向在自然语义边界断开，同时可控上限	工程最常用的基础策略 （LangChain 的 <code>RecursiveCharacterTextSplitter</code> ），普通文本场景的好默认
语义切分 Semantic	按句子分割，计算相邻句子的 Embedding 相似度，相似度明显跌落处 = 话题转换处，在此切分	语义完整性最好，chunk 内主题聚焦	需对全文做 Embedding，速度慢成本高；chunk 大小不规则。适合质量要求极高、话题切换频繁的场景
结构感知 Structure-aware	按文档层级结构切：Markdown 按 <code>###/####</code> 标题、法律文件按条款编号、代码按函数/类边界、PDF 按页码+章节标题	每个 chunk 天然对应一个有意义主题， 检索精准度往往最高	有明确层级结构的文档的最优选择

3.3 Overlap：被低估的关键参数

问题：一句话恰好落在两个 chunk 边界——前半句在 chunk 1，后半句在 chunk 2。单独检索到任何一个，拿到的都是一半，信息不完整，答案就会出错。

做法：相邻 chunk 保留一段重叠内容。如 chunk 512 token、overlap 64 token，则 chunk 2 的前 64 token 与 chunk 1 的后 64 token 完全相同。

效果：边界附近信息在两个 chunk 里都有备份，命中哪个都能拿到完整上下文。

3.4 Chunk 过大、过小的问题与经验值

- 过小：**语义不完整（如只检索到"是的，这种做法符合规范"——符合什么规范完全不知道）；chunk 数量暴增，索引膨胀、检索变慢。

- **过大**：话题太多，相关度被稀释（chunk 2000 token，用户只关心其中 50 token，但 2000 token 全塞进 Context，注意力难聚焦）；chunk 越大向量表示越"平均化"，语义越模糊，检索精准度越低。

经验参考值（无通用最优，需按业务调整）：对话型问答 256~512 token；知识库检索 512~1024 token；长文档摘要 1500~2000 token。

3.5 策略选择决策框架

- 文档有清晰结构（章节、标题、条款）→ **结构感知切分**
- 普通散文、无明确结构 → **递归字符切分**，配合合适 chunk size 和 overlap
- 质量要求极高、话题切换频繁 → **语义切分**（接受计算成本）
- 快速验证原型 → **固定长度**够用，记得上线前换掉

面试 Q：你们项目里 chunk 是怎么切的？

先说策略及原因 → 再说具体参数（chunk size、overlap）→ 最后说如何评估调整。示例："我们的产品手册有章节结构，所以用结构感知切分、按二级标题划分。每个 chunk 控制在 600 token 内，overlap 设 80 token。后来发现部分技术规格内容跨段落，把 overlap 调大到 120 token 后，相关问题的召回率有明显提升。"

面试 Q：为什么要设置 overlap？

防止重要信息落在 chunk 边界被割裂，通过相邻 chunk 共享一段内容，保证边界附近信息在两侧都能被完整检索到。

四、Embedding：语义压缩、选型与 Rerank 的区别

4.1 Embedding 到底是什么

Embedding 是将文本（或图片、音频）映射到**高维向量空间**的技术。一个句子经 Embedding 模型处理后变成几百到几千个浮点数组成的数组，如 `[0.23, -0.87, ..., 0.56]`（维度通常 384、768 或 1536 维）。关键不在数字本身，而在**位置**：语义相似的文本向量在空间里彼此距离近，语义不相关的距离远。这是 Embedding 的核心承诺：**用空间距离来表达语义相似度**。向量检索的本质，就是计算 Query 向量与所有文档向量的距离，取最近的几个。

为什么关键词搜索救不了语义匹配：用户问"飞机"，文档里写的是"民用航空器"——加再多同义词词库也没用，因为传统搜索在**拼文字**，而 Embedding 让机器匹配**语义**。同一个语义换各种说法，好的 Embedding 模型给出的向量应非常接近。

不是生成文本，是在压缩语义：生成式大模型（GPT/Claude）目标是输出文字；Embedding 模型输出的是向量表示，把一段文字的"语义"压缩到固定维度的数字数组里，保留语义信息、丢掉格式和措辞细节。

4.2 模型选型四维度

维度	要点
向量维度	维度越高语义表达越细腻，但存储开销和检索速度相应增加。常见：384 维（轻量）、768 维（均衡）、1536 维（高精度）。中等规模业务 768 维是合理起点。
最大 token 限制	超过上限会被截断，直接约束 chunk 最大大小。如 <code>bge-small-zh</code> 支持最多 512 token，chunk 就不能超过它。 选模型前一定要核对这个参数。
中文支持	中文 RAG 场景最关键的选型依据。通用英文模型对中文支持明显偏弱（如 OpenAI <code>text-embedding-ada-002</code> ）。中文场景口碑较好的开源选项： BGE 系列、M3E 系列 。
非对称检索支持	RAG 典型非对称场景：Query 是短问题，文档片段是长说明。有些模型专门优化了这类 Query-Document 对，如 BGE 系列支持给 Query 加特殊前缀 <code>"Represent this sentence for searching relevant passages:"</code> 提升检索效果。

4.3 中文场景 vs 通用场景

- 分词边界：**中文没有天然空格分词，字符级语义表达与英文差异显著，通用模型对中文语义捕捉明显弱于专门训练的中文模型。
- 专业术语：**中文业务里大量出现行业专业词（法律条款、医疗术语、金融名词），通用模型往往无法精准表达其语义关系。
- 评估基准：**MTEB（Massive Text Embedding Benchmark）中文 Retrieval 子任务榜单是重要参考。开源中文模型中 BGE 系列（`bge-large-zh-v1.5`、`bge-m3` 等）和 E5 系列表现出色。
- 选型建议：**中文业务优先 BGE / M3E；需要中英双语检索选 `bge-m3`（专门优化多语言检索）。

4.4 Embedding vs Rerank（高频面试题）

一句话：Embedding 负责"粗筛"，Rerank 负责"精排"。核心差异在建模方式：

维度	Embedding（Bi-Encoder 双塔）	Rerank（Cross-Encoder 交叉编码）
编码方式	Query 和每个文档片段各自独立编码成向量，用向量距离（余弦相似度）衡量相关性	把 Query 和每个候选片段拼接在一起整体输入模型，直接输出相关性分数
能否预计算	文档向量可离线预计算存入向量库，检索时只算 Query 向量 + 近似最近邻	无法预计算，每次逐对推理
速度 / 规模	极快，毫秒级从百万量级文档召回	慢，只适合对少量候选（top-20 以内）精排
精度	较粗，语义相似但未必精准回答	能看到 Query 与文档的完整交互，语义理解更深、精准度更高
角色	从全量文档快速召回 top-K（可能 20-50 个）	从候选里精选最相关的 top-N（通常 3-5 个）传入 Context

两者是流水线关系，不是二选一。

4.5 常见误区

- "模型越大越好"：不一定。更大通常精度更高但推理更慢、内存更大。实时检索场景，轻量但针对检索优化的模型（如 `bge-small-zh`）可能效果更好、成本更低。
- "用 OpenAI 的就够了"： `text-embedding-3-small/large` 英文极好，但中文场景建议评测对比；用专门中文模型还减少第三方依赖、降低成本。
- "Embedding 和 Rerank 随便配"：Embedding 选择无强制约束，但 Rerank 模型最好与 Embedding 模型在训练数据分布上相近，否则评分口径不一致，精排效果不稳定。
- "换 Embedding 模型不用重建索引"：完全错误。不同模型的向量处于不同语义空间，旧索引向量与新 Query 向量根本不可比。换模型必须全量重新 Embedding、重建整个向量索引。

面试 Q：Embedding 模型怎么选？

四个维度：① 语言场景——中文优先中文优化模型（BGE、M3E）；② 输入长度——最大 token 限制决定 chunk 上限；③ 向量维度——按存储与精度权衡，768 维是合理起点；④ 任务类型——非对称检索（短 Query 找长文档）与对称检索最优模型不同，看具体业务。

面试 Q：Embedding 和 Rerank 有什么区别？

Embedding（双塔/Bi-Encoder）对 Query 和文档分别独立编码成向量，通过向量相似度召回，可离线预计算、检索极快、可处理百万量级，适合粗筛；Rerank（Cross-Encoder）把 Query 和候选文档拼接后联合输入，直接输出相关性分数，能看到深度交互、精准度更高，但无法预计算、需逐对推理，只适合少量候选精排。流水线关系：Embedding 粗筛候选池，Rerank 精排出最终上下文。

五、向量数据库：为什么不用 MySQL、ANN 索引原理

5.1 向量数据库要解决什么问题

场景：100 万个文档 chunk，每个是 768 维向量；用户提问向量化后，要找到 100 万向量里最相似的 top-5。最直接的方法是暴力搜索：逐一计算余弦相似度再排序——普通服务器上需要几百毫秒到几秒，对实时对话完全不可接受；向量数增长到千万、亿级时耗时线性增长，无法用于生产。**核心问题：在海量高维向量中，如何毫秒级内找到最相似的结果？**

5.2 为什么不能用 MySQL 做向量检索（高频）

- 功能层面：**传统关系型数据库的查询本质是精确匹配或范围扫描（如 `WHERE age > 25`），靠 B-Tree/Hash 索引快速定位。向量相似度搜索问的是“哪些向量和我的查询向量**最接近**”——高维连续空间里的几何问题。传统索引结构设计之初没有考虑“高维几何相似度”，对这类问题完全无效。
- 性能层面：**即便使用扩展支持向量存储（如 PostgreSQL 的 pgvector），大规模下全量计算余弦相似度的复杂度是 $O(N)$ ，远无法满足毫秒级需求。向量数据库通过 HNSW、IVF 等近似索引把复杂度降到近似 $O(\log N)$ 。

5.3 ANN：在速度和精度之间取得平衡

向量数据库不做暴力全量计算，而是预先构建索引结构，检索时只访问一小部分向量就能找到近似最近邻——这就是 ANN（Approximate Nearest Neighbor，近似最近邻搜索）。注意“近似”二字：不保证找到绝对最近邻，但实践精准度足够高（通常 95%+，通过 recall@K 衡量），速度提升是数量级的。最主流的两类索引算法：

算法	原理	优缺点
HNSW 分层可导航小世界图	构建层级图：最顶层少量节点形成“高速公路”，底层所有节点形成“街道网络”。检索从顶层入口进入，沿邻居边快速导航、逐层下降，像 GPS 导航一样越来越精细，定位到目标附近的最近邻	优点：查询速度极快、精度高，是 Milvus/Qdrant/Weaviate 等主流向量库的默认索引。代价：建索引时内存占用较大
IVF 倒排文件索引	先对所有向量做 K-Means 聚类，相似向量分组到同一“货架”；检索时先找 Query 最近的几个货架，只在这些货架内精细查找	优点：内存占用低，适合超大规模数据集。缺点：Query 落在两个货架边界时可能漏掉相关结果（边界效应）

超参数权衡：HNSW 有 M（每个节点最大连接数）、efConstruction（建图搜索深度）等参数——越大精度越高，但内存占用和构建时间成比例增加。需要按场景找精度/速度/内存的平衡点，不是越大越好。

5.4 向量数据库的附加能力

- 元数据过滤（Metadata Filtering）：**每个向量附带结构化元数据（来源、时间、部门、类型），检索时叠加条件——“只搜最近三个月”“只搜产品部门”，实现语义相关 + 业务过滤的组合查询。

- **混合检索 (Hybrid Search)**：同时跑向量搜索和关键词搜索，用融合算法（如 RRF, Reciprocal Rank Fusion）合并。对中文专业术语场景尤其有用——有些专有名词的语义表示很弱，必须靠关键词精确匹配。
- **向量更新和删除**：知识库持续更新时索引也要对应更新；不同向量库对此的支持成本差异很大，选型要考虑业务更新频率。

5.5 主流向量数据库选型

类别	代表	适用
专用向量数据库	Milvus、Qdrant、Weaviate、Pinecone	以向量检索为核心设计，功能最完善、性能最优，适合大规模生产；Pinecone 适合云端托管免运维
传统数据库向量扩展	PostgreSQL + pgvector、Elasticsearch + dense vectors	已有基础设施的小规模场景可用；百万量级向量时性能明显不如专用方案
轻量级本地库	FAISS、Chroma	FAISS 是 纯向量索引库（不是完整数据库） ——没有持久化、元数据管理、CRUD，只负责索引构建和搜索计算，适合原型或嵌入应用；Chroma 是轻量级向量库，适合开发调试

选型决策：几十万向量以内 → Chroma 或 pgvector 足够；百万以上、需要元数据过滤和混合检索 → Milvus 或 Qdrant；要云端托管免运维 → Pinecone。

5.6 常见误区

- **"ANN 不精确所以不可靠"**：实践中召回精度通常 95%–99%（recall@K 衡量），对 RAG 完全够用，而速度收益是数量级的。
- **"相似度高就代表语义相关"**：相似度是 Embedding 模型学出来的，取决于模型训练质量和领域覆盖。模型对某领域支持不足时，相似度高语义也可能不相关。**向量库只是工具，底层模型质量才是决定因素。**
- **"用 FAISS 就是用了向量数据库"**：FAISS 没有持久化、元数据、CRUD，生产环境需自己封装或直接用完整向量数据库。

面试 Q：为什么不用 MySQL 做相似检索？

两层：功能层面，B-Tree 等索引无法支持"高维空间近似最近邻"这种几何问题；性能层面，即便 pgvector 等扩展，百万级向量全量计算复杂度 $O(N)$ ，无法满足毫秒级。向量库用 HNSW/IVF 把复杂度降到近似 $O(\log N)$ 。

面试 Q：向量数据库为什么能加速召回？

核心是 ANN：预建索引避免全量计算。以 HNSW 为例，分层可导航图让检索从顶层稀疏图入口快速导航、逐层定位，只访问极少数节点即得近似最近邻，比暴力搜索快几个数量级；代价是"近似"，但 recall 95%+ 足以满足 RAG 需求。

六、RAG 优化思路全景

6.1 优化要对症下药

先建立认知：**RAG 优化没有万能药**。"加个 Rerank 就搞定""换更好的模型就行"很多时候是用锤子找钉子。正确姿势：**先诊断问题出在哪个环节，再选择对应优化手段**。优化手段按链路位置分三类：Query 侧（问题进检索前处理）、检索侧（改善召回质量）、Context/生成侧（改善最终喂给 LLM 的上下文）。

6.2 Query 侧优化

- **Query 改写 (Query Rewrite)**：用户原始问题口语化、语义模糊，不一定适合直接向量检索。用 LLM 先处理一轮——扩展同义词、修正表述、补全隐含条件，让改写后的查询贴近知识库文档的用词习惯。例："这个合同能退吗" → "合同解除条款、退款政策、合同撤销条件"，多角度检索覆盖率明显更好。
- **Multi-Query (多角度查询)**：一个问题可多角度理解时，生成多个不同版本分别检索，多路结果合并去重。例："我们产品的 SLA 是多少" → 展开为"服务等级协议内容""故障响应时间""可用性承诺""赔偿条款"四路检索，召回率显著提升。

6.3 检索侧优化

- **混合检索 (Hybrid Search)**：纯向量检索擅长语义理解，但对专有名词、精确匹配（人名、产品型号、法规编号）不如关键词搜索。混合检索同时跑向量检索 + BM25，用融合算法合并——最常见的是 **RRF (Reciprocal Rank Fusion)**：一个文档在两路结果里排名都靠前，融合分数就高；只在一路靠前的打折。RRF 对超参数不敏感、实现简单、效果稳定，是工程首选。
- **元数据过滤 (Metadata Filtering)**：在相似度检索同时叠加结构化条件（"只搜 2024 年之后""只在合同类文档里"）。不改变检索算法，但大幅缩小范围——既提高精准度，又降低计算开销。

6.4 精排优化：Rerank

位置很具体：向量检索召回 top-K 之后，用 Cross-Encoder 对每个 (Query, Chunk) 对精细打分、重新排序，只保留真正高相关的 top-N 传给 LLM。**核心价值**：向量检索的相似度是独立编码的，不能完整捕捉 Query 与文档间的深度语义关联；Cross-Encoder 能看到两者完整交互，打分更精准。**混合检索 + Rerank 是目前工程上最成熟的检索优化组合**：两路各召回候选 → RRF 融合成候选池 → Cross-Encoder 精排 → 最终 3-5 条高质量 chunk 进入 Context。

Rerank 什么时候该加：Recall@K 已经足够好（相关内容确实被召回了），但答案精准度仍不高——召回了相关内容但没精确筛出最核心的部分。

什么时候不急着加：Recall@K 本身很低，相关内容根本没被召回——加 Rerank 没有意义，应先解决召回问题。

6.5 Context 侧优化

- **父子块检索 (Parent-Child Retrieval)**：解决 Chunking 经典矛盾——小 chunk 利于精准匹配但上下文不完整，大 chunk 上下文完整但精准度差。做法：两级索引，用小 chunk (子块) 做检索，实际传给 LLM 时用包含它的大 chunk (父块)。例：2000 字文章按段落切 5 个 400 字子块做索引，命中子块后传整篇文章（或更完整的部分）。
- **Context 压缩 (Context Compression)**：召回 chunk 含大量无关内容时，用一个 LLM 先阅读并提取与 Query 高度相关的句子、丢弃无关内容，再拼最终 Context。显著减噪，但额外增加一次 LLM 调用的延迟成本，适合对准确性要求极高、对延迟容忍度较高的场景。

6.6 初学者优化方法论 (重要)

1. **先建 baseline，做好评估**：优化前先用 Recall@K 和答案质量指标 (LLM 打分或人工评估) 建立可量化基线。没有基线，无法判断优化是否有效。
2. **优先解决“能不能找到”，再解决“找到了对不对”**：Recall@5 很低时先优化召回（换 Embedding、调 Chunk 策略、加混合检索），而不是先加 Rerank。
3. **每次只改一个变量**：同时改 Chunk 大小、Embedding 模型和 Rerank，永远不知道哪个起了作用。
4. **按成本/收益定优先级**：混合检索（加 BM25 索引）成本低覆盖广，通常是性价比最高的第一步；Query 改写、Rerank 增加延迟成本，按业务 SLA 决定。

面试 Q：你们 RAG 系统做了哪些优化？

按 Query 侧/检索侧/精排侧/Context 侧分类回答，说清每个优化针对的问题、实施前后效果、以及取舍。要有数据有逻辑：“我们发现 Recall@5 在 60% 左右，先引入混合检索提升到 78%；再加 Rerank，答案精准率提升约 15%。”

面试 Q：混合检索怎么实现？两路结果怎么融合？

向量检索用 Embedding + 向量库；关键词用 BM25 (Elasticsearch 或向量库关键词功能)。融合用 RRF：每个文档把在两路的名次取倒数相加得融合分、重新排序。RRF 对超参不敏感、实现简单、效果稳定。

七、长文档与代码检索

7.1 三大硬问题与核心原则

几百页的需求文档、几十万行的代码仓库，"512 Token + overlap"只适合原型。多了三个硬问题：**结构不能断、版本不能错、权限不能漏**。真正要解决的不是"Chunk 设多大"，而是：**怎么用小块找准，再按结构拿回完整、当前且有权限的证据**。

一句话原则：检索单位不等于返回单位；相关不等于可用；可用必须满足**完整、当前、有权限**。

7.2 长文档：结构化切分

需求文档通常是"章节 → 小节 → 条款 → 表格"的层级。某个子块里只有"退款周期为 7 个工作日"，但适用范围写在父标题"企业版年付客户"里——只返回子块答案缺条件，返回整篇噪声和 Token 太多。所以长文档不能只存文本，还要保存它在**标题树中的位置**，例如 `产品手册 / 计费规则 / 企业版 / 退款政策`。这条路径既参与检索和 Rerank，也能在命中后帮系统找到父章节、相邻条款和表格说明。

7.3 代码：AST 切分

代码的自然边界不是句号和换行，而是**模块、类、函数、方法和语句块**。固定切片从函数中间切开，chunk 里可能只有实现没有函数名、参数和返回类型；也可能只有调用点没有定义。向量能判断"语义相似"，却不知道片段属于哪个符号。**更合适的做法：先用解析器生成语法树，再按 AST 节点切分**（Tree-sitter 可为源码构建具体语法树并增量更新）。工程中通常保留：

- 函数/方法签名、注释和函数体
- 所属类、模块、文件路径和符号全名
- 起止行号、语言、导入关系和被调用符号
- 仓库、分支、提交哈希与内容哈希

注意：AST 负责"不把结构切坏"，不负责解决所有检索问题——README、配置、SQL、错误日志仍要用对应的结构解析器或文本切分策略。

7.4 Parent-Child：找得准 + 读得懂

建立两级或多级节点：

节点	粒度	职责
Child	函数、短条款、小段落	用于 Embedding、关键词召回和 Rerank
Parent	类、章节、完整规则	用于最终返回给模型补齐上下文，每个 Child 保存 <code>parent_id</code> ，父节点存文档存储
Document	文件、整篇文档	只在跨章节任务中按需继续向上扩展

命中 Child 后不是机械返回整篇文档，而是按问题取父节点、标题路径、必要的相邻节点。**父级扩展是预算决策，不是固定动作**：函数 30 行可返回整个函数；类有 3000 行就只补签名、字段、目标方法和直接依赖；章节两页可返回完整章节，几十页就补标题路径和相邻条款。可设三个门槛：**父块最大 Token、允许扩展的层级、相邻节点数量**。超预算时优先保留定义、约束、输入输出和直接依赖，不要粗暴截断末尾。

与 overlap 的区别：Overlap 只缓解相邻切片的边界断裂，不知道标题、类、函数的真实父子关系；Parent-Child 是显式结构关系，可跨多个子块补回完整父级，但索引和存储更复杂。

7.5 版本与权限必须进索引

代码不是静态文本：同一路径在 `main`、功能分支和历史 commit 上可能完全不同。分支随提交移动，而 commit 表示确定的快照。索引至少要保存：

```
{ "repo": "payment-service", "path": "src/refund/RefundService.java", "symbol":  
  "RefundService.apply", "branch": "feature/refund-v2", "commit": "a1b2c3d",  
  "parent_id": "class:RefundService", "acl": ["team:payment"]} }
```

在线查询时，用户问题必须和当前工作区范围一起进入检索：目标仓库、当前分支或 commit、调用者所属租户和权限组。

先过滤，后相似度召回。如果先召回再删权限不匹配的结果：受限内容仍可能进入候选缓存、Rerank 服务或调试日志；且删除后候选数量不足，召回质量突然下降。访问控制要在**检索边界**生效。

7.6 在线检索链路（七步）

1. 从会话或 IDE 获取 `repo + branch/commit + user/tenant`
2. 解析 Query，识别符号名、文件名、错误码和自然语言意图
3. 先做版本、路径和 ACL 过滤
4. 同时跑向量检索与 BM25，合并候选
5. 在 **Child 粒度** Rerank，避免大父块稀释相关度
6. 根据 `parent_id` 取父级、标题路径或必要相邻节点
7. 去重并按 Token 预算组装 Context，附带路径、行号和 commit 作为引用

最容易写反的是第 5、6 步：先在小块上排序，再向父级扩展。如果先把所有 Child 展开成大块再 Rerank，相关信号会被大量父级文本冲淡，延迟也会上升。

7.7 落地与评估

- **最小方案建 Baseline**：结构化切分、Child 混合检索、Parent 补全、版本和 ACL 前置过滤。不要一开始就加调用图、摘要树和复杂 Agent。
- **评估集覆盖五类问题**：跨小节答案、精确符号查找、同文件不同分支、历史 commit、无权限访问。
- **专有指标**：父级补全正确率、版本错误率（是否引用目标 commit 之外的内容）、**越权召回率（目标必须是 0）**、Context 有效率（进入上下文的 Token 中多少真正支持答案）、索引新鲜度（代码更新到

可检索的延迟)。

- **增量更新**：用 `commit + path + content_hash` 判断新增/修改/删除，只重建受影响节点及父子关系。全量重建在代码库大时成本和更新延迟都会失控。

拓展 Q：有了 AST 切分，还需要关键词检索吗？

需要。函数名、类名、错误码、配置键和路径是精确符号，BM25 或符号索引通常比纯向量更稳；自然语言描述和业务意图交给向量检索。两路召回后统一 Rerank。

拓展 Q：代码依赖要全部展开进 Context 吗？

不是。调用图会迅速扩散。先返回目标符号，再按问题补一跳直接依赖；只有测试失败分析、跨模块重构等任务才逐层扩展，并设深度与 Token 上限。

拓展 Q：多分支会让索引膨胀吗？

会。常见做法：长期保留默认分支和活跃分支，历史版本按需构建或设过期策略；相同内容按内容哈希去重，但版本、路径和 ACL 关系不能丢。

八、Agentic RAG：从固定流水线到动态检索循环

8.1 定义

Agentic RAG 不是某个固定算法，也不是某个框架的专属功能，而是一类架构模式。在这类系统里，大模型不只负责最后写答案，还参与决定：要不要检索、问题怎么拆、查哪个数据源、检索结果是否可用、要不要换 Query 再查、什么时候停止。核心转变：**传统 RAG 把检索写死在流程里，Agentic RAG 把检索变成 Agent 可以按需使用、反复使用的工具。**

8.2 演进四阶段（每步都因上一步解决不了新问题）

阶段	谁决定检索路径	检索次数	结果不好怎么办
传统 RAG (2020 论文)	开发者写死	通常 1 次	带着错误结果继续生成
Advanced RAG	开发者写死更多优化 (Query 改写、Multi-Query、混合检索、Rerank、父子块、Context 压缩)	固定若干次	依赖预设的改写、重排策略
自适应/纠错 RAG Self-RAG / CRAG / Adaptive-RAG	分类器或模型参与判断	0 到多次	评估、改写或补充检索
Agentic RAG	Agent 根据目标和状态决定	动态	换 Query、换工具、补证据或拒答

传统 RAG 的隐含假设：“用户的一句话已经足够适合检索，且第一次返回的 Top-K 就包含答案”——现实里经常不成立。Advanced RAG 让流水线变强，但**流水线本身仍是固定的**：简单问题照跑全流程，复杂问题一次没查全也不会自己增加下一轮。第三阶段的关键思路：Self-RAG（按需检索+对检索内容和生成结果反思）、CRAG（先评估检索结果，不可靠时触发纠正动作）、Adaptive-RAG（按问题复杂度在不检索/单步/多步间选择）——检索从“必走一次的固定步骤”变成“可以判断、评估和纠错的过程”。第四阶段：有了 Function Calling、ReAct 和 Agent 运行时，检索被包装成工具，Agent 接到的不再是一个待搜索的 Query，而是一个需要完成的目标。

8.3 Agentic RAG 解决什么问题（与优化手段不同层）

混合检索和 Rerank 解决的是：这一次怎么查得更准。

Agentic RAG 解决的是：这次该不该查、该查几次、查什么、结果不够时下一步怎么办。

1. **一个 Query 装不下复杂问题**："对比两个版本并找出地区例外"至少含版本检索、差异比较、地区过滤三个子问题。整句向量化可能每个方向都沾一点但哪个都查不准。Agent 先拆子问题、分别找证据、再合并。
2. **第一次检索错了，固定流程不会自救**：传统 RAG 召回不相关文档后，Rerank、Prompt、大模型都只能在错误候选集里挣扎。Agentic RAG 先评估证据：相关性低、来源冲突或关键字段缺失，就重写 Query、扩大范围、换数据源，或明确告诉用户证据不足。
3. **不同问题需要不同数据源**：政策解释查文档库、历史订单查数据库、刚发生的变化查网页或业务 API。单一向量库不是企业全部知识，Agent 的价值之一是在多个受控工具间做路由。
4. **不是所有问题都值得检索**："把上一段改得更口语化"不需要查知识库。每次都跑改写+检索+Rerank 只会增加延迟、Token 和费用。Agent 可判断直接回答或走更轻的路径。
5. **多跳问题需要带着新线索继续查**：第一轮查到"新政策 7 月生效"，第二轮才知道查"7 月之后的订单"，第三轮再核对"华东区例外"。后一次检索依赖前一次结果，预写死的 Query 做不到。

8.4 核心原理：五个角色 + 一个循环

1. **规划与路由**：判断任务复杂度，决定直接回答、单次检索还是拆成多步
2. **检索工具**：向量检索、BM25、数据库、网页搜索、业务 API，各自暴露清晰能力
3. **状态记录**：保存子问题、已查来源、候选证据、冲突点、剩余预算
4. **证据评估**：检查相关性、完整性、来源可靠性和相互一致性
5. **停止与生成**：证据足够就组织带引用的回答；证据不足且预算耗尽就拒答或请用户补充

理解目标 → 决定：直接回答还是调用检索工具？ → 拆分子问题 → 选择数据源 → 执行检索
→ 评估证据：相关吗？完整吗？互相冲突吗？
→ 不够：改写 Query / 换工具 / 继续检索；足够：整合证据 / 标注来源 / 生成答案

这就是 **ReAct 在检索场景的落地**：模型基于当前状态决定行动，工具返回观察结果，模型再决定下一步。

"反思"不能只靠模型拍脑袋说证据够了。生产系统通常还要加确定性规则：至少需要几类证据、是否命中指定版本、引用能否回到原文、数值是否来自结构化数据、最大允许几轮、总耗时和费用上限。**Agent 负责动态决策，规则负责兜底，缺一不可。**

8.5 完整例子

用户问："对比 2025 和 2026 两版差旅报销政策，并说明上海分公司的特殊规定。"传统 RAG 整句检索一次，如果召回全是 2026 版，总结再流畅也没完成"对比"。Agentic RAG 会：

1. 识别三个证据槽位：2025 总部政策、2026 总部政策、上海分公司补充规定
2. 为三个槽位分别生成 Query，可并行检索减少串行等待
3. 检查版本号、生效日期、适用组织和来源权限，不只看向量相似度
4. 缺 2025 版就去历史归档库继续查；两个来源冲突就优先核对正式发布记录
5. 三类证据齐全后做差异比较，每条结论带回原始来源

真正的变化不是"多查了几次"，而是系统知道自己为什么查、还缺什么、什么时候算查完。

8.6 关键辨析：Agentic RAG ≠ 多 Agent

一个 Agent 配几个检索工具，加上明确的状态机和证据检查，已经是 Agentic RAG。只有任务确实存在稳定职责边界（如法规检索、财务数据查询、证据审计需要不同权限或不同模型）才值得拆多 Agent；否则多 Agent 带来更多通信、上下文复制、错误传播和调试成本。先把单 Agent 的检索循环跑稳，再考虑多 Agent。

8.7 落地权衡：更强，但也更难上线

维度	传统 RAG	Agentic RAG
路径	固定，容易预测	动态，每次可能不同
延迟	低且稳定	多轮工具调用，波动更大
成本	容易估算	与规划轮数、检索次数相关
调试	看固定链路	要看完完整决策轨迹和工具结果
权限	一个检索入口较简单	多数据源需要逐工具鉴权
上限	适合简单、单跳问答	适合复杂、多跳、多源任务

落地六条守则：① 限制循环（最大步骤、超时、Token 和工具调用预算，防止越查越远）；② 工具要窄（用途清晰、参数 Schema 和返回值稳定，别做"万能搜索"）；③ 权限前置（检索继承用户权限，不能绕过原系统访问敏感数据）；④ 证据可回放（记录每次 Query、命中文档、评分、选择理由和最终引用）；⑤ 失败能降级（超时或不稳时退回固定 RAG、澄清问题或拒答）；⑥ 分层评估（既评检索找全没有，也评答案是否忠于证据；统计平均步骤、P95 延迟、单次费用）。

8.8 适用场景与升级顺序

- 适合 Agentic RAG：多跳推理（后一步依赖前一步的新线索）、一个请求含多个子问题、知识分散在文档库/数据库/网页/业务 API、对证据完整性和冲突处理要求高、问题模糊需先澄清意图。
- 固定 RAG 更划算：FAQ/产品手册等单跳问答、数据源单一命中率高、首字延迟要求严格、Workflow 就能可靠完成的明确规则流程。
- 合理升级顺序：先把传统 RAG 的文档、切片、检索和评估做好 → 再增加路由与纠错 → 最后才开放更动态的 Agent 循环。如果基础召回都做不好，套一层 Agent 只会让系统用更贵、更慢的方式找不到答案。

面试参考话术：Agentic RAG 和传统 RAG 有什么区别？

"传统 RAG 的检索路径由开发者预先写死，用户问题经过一次或固定次数检索后直接生成。Agentic RAG 把检索封装成 Agent 工具，由模型根据任务状态决定是否检索、如何拆分 Query、选择哪个数据源，并对中间证据做评估；证据不足时可以改写或继续检索，足够时才停止并生成带引用的答案。它更适合复杂、多跳和多数据源问题，但会增加延迟、成本、可观测性和权限治理难度。"

定义、原理、场景和代价都有了，比只说"Agentic RAG 更智能"有用得多。**Agentic RAG 真正增加的**不是一层 Agent 外壳，而是检索过程中的决策权、反馈和停止条件。

九、RAG 排障诊断：检索侧五类 + 生成侧四类

9.1 总原则

很多人一碰到 RAG 答不准，第一反应是换模型、上 Rerank，却从不拆解根源。事实上 RAG 就两段关键链路：离线构建索引、在线检索生成。答案不好，要么是检索崩了（没找到、找错了），要么是生成崩了（找到了但没用好）。不先定位是检索还是生成的问题，所有优化都是治标不治本。

9.2 检索侧五类常见问题

问题	机制与典型症状	应对思路
1. 文档质量差	最根本也最易被忽略。扫描件 OCR 乱码、格式混乱的 PDF、充满模板噪声的 HTML——再好的 Embedding 也无法从噪声提取语义。症状：检索结果出现"第 3 页""版权所有"....."类片段	Chunking 之前加强预处理：清除页眉页脚、过滤模板噪声、低质量文档专项清洗甚至人工校对
2. Chunk 粒度不合理	太小：片段只有一两句话，语义不完整，向量无法准确表达主题，命中了相关话题但内容不完整；太大：多主题混合，向量表示与 Query 匹配模糊，召回精准度下降	调整切分策略与参数（见第三章）
3. Embedding 选型不当	模型对中文或专业领域覆盖不足，语义相似的内容在向量空间距离很远，检索根本找不到。症状：同样意思换说法后检索结果差异巨大、同义句得分很低	候选模型评估（MTEB 中文榜参考），在业务数据上跑召回率对比实验，选最适合当前领域的
4. 召回不足	文档更新但索引未及时重建、部分文档入库时被误过滤、top-K 太小相关内容排在 K 之外。症状：知识库里有答案但系统给不出；手动搜索能找到、自动检索找不到	检查索引覆盖是否完整；适当增大 top-K（如 5 → 10-20）配合 Rerank 过滤；引入混合检索补充
5. 缺少 Rerank	向量相似度和"真实相关性"是两个维度——有些片段向量很近（讨论相关话题）但未必精准回答具体问题。症状：答案"相关但不精准"、方向对细节错；Context 里明明有答案但拼接的回答不准	召回后加 Cross-Encoder Rerank，对 (Query, Chunk) 对精细评分，只保留真正高相关的结果

9.3 生成侧四类常见问题

问题	机制与典型症状	应对思路
----	---------	------

1. Prompt 拼接有问题	chunk 之间无分隔符、来源信息缺失、指令和内容混在一起、指令表述模糊。症状：模型混淆多个文档内容；"资料中没有答案请说不知道"的指令遵从率很低	明确区分"系统指令"和"参考资料"，不同来源加分隔符，指令里明确说明资料不足时如何处理
2. 上下文干扰 (Context Noise)	低相关 chunk 混入 Context 稀释注意力，模型在大量无关信息中"迷失"。无 Rerank 的系统尤为突出，也与 top-K 过大有关。症状：答案混入奇怪的无关细节；给出综合多个不相关来源的"平均答案"	Rerank 过滤低相关 chunk；减少传入 chunk 数量（宁少勿滥）；Prompt 明确要求只根据提供的资料作答
3. 缺少引用约束	Prompt 没有限制"只基于资料作答"，模型把参数知识与检索内容混合回答——引入幻觉，答案看起来更完整但准确性存疑	系统 Prompt 加明确引用约束："请严格根据以下资料回答，资料中没有的内容请明确说明无法回答"，并要求标注来源
4. 模型理解能力不足	检索质量很高但模型推理成为瓶颈——尤其答案需跨多个 chunk 综合推断或问题本身需复杂逻辑推理时。小模型尤为明显	升级到推理能力更强的模型；Prompt 明确要求逐步推导；或通过 Query 拆解把复杂问题分解成多个简单检索任务

9.4 系统诊断三步法

1. **判断检索问题还是生成问题：**手动把用户问题直接查向量库，看返回的 top-K chunk 里有没有正确答案。有答案但回答错 → 生成侧问题；没有 → 检索侧问题。
2. **检索侧进一步定位：**召回率评估——对一批"已知有答案"的问题统计正确 chunk 是否出现在 top-K (Recall@K)。Recall@K 低 → 分析是 Embedding 问题（语义匹配差）还是文档覆盖问题（内容没入库）。检索结果质量评估——看 top-K 相似度分数分布：分数普遍很低 → Embedding 语义空间不匹配；分数高但内容无关 → Embedding 在当前领域理解有偏差。
3. **生成侧进一步定位：**把正确 chunk 手动构造 Prompt 喂给模型。能正确回答 → 是检索问题（找对了内容模型能用好）；不能 → Prompt 设计或模型能力问题。

经验值：Recall@5 低于 70%，说明检索侧有明显问题，需要优先优化。

面试 Q：你们 RAG 系统答不准，你会怎么排查？

先从检索侧、生成侧分析：手动查看 top-K 检索结果——答案在里面 → 生成侧问题（Prompt、上下文噪声、引用约束、模型能力）；不在 → 检索侧问题（召回率低、Embedding 选型、Chunk 质量、文档覆盖）。然后用 Recall@K 等指标量化，再针对性调整。

面试 Q：如何评估召回效果？

构造评估集——一批"已知有答案的问题"及对应正确 chunk，计算 Recall@K（K 通常取 5 或 10）。Recall@5 低于 70% 说明检索侧有明显问题，优先优化。

十、RAG 评估体系

10.1 为什么必须有评估体系

很多团队做 RAG 优化靠感觉：“换个 Embedding 试试 → 感觉好点了 → 上线”。问题在于：① 无法量化优化效果（提升 5% 还是 50%？哪些场景变差了？）；② 无法对比不同方案；③ 无法发现退化（加了新功能后老场景有没有变差？）；④ 面试答不上（“感觉更好了”不是工程师的回答）。真正的工程化 RAG，必须先建立评估体系，再做优化。这是应用开发和“调 API 玩玩”的本质区别。

一句话：检索侧保证能找到，生成侧保证用得好。没有评估的优化，就是拿着锤子找钉子。

10.2 检索侧四大指标

指标	定义与公式	工程意义与局限
Recall@K 召回率	top-K 结果中正确答案片段出现的比例 $\text{Recall@K} = \frac{\text{检索到的相关 chunk 数}}{\text{所有相关 chunk 总数}}$ 例：知识库有 3 个 chunk 含答案，top-5 包含其中 2 个 → $\text{Recall@5} = 66.7\%$	衡量“相关内容有没有被找到”，检索系统第一道防线。 Recall<70% 说明检索侧有严重问题 ，加 Rerank 也救不回来。局限：只看“找没找到”，不管“找到多少垃圾”——top-100 全含答案但 99% 是噪声会严重干扰生成
Precision@K 精准率	top-K 中相关内容占的比例 $\text{Precision@K} = \frac{\text{检索到的相关 chunk 数}}{K}$ 例：top-5 有 2 个相关 → 40%	低说明噪声太多，会稀释模型注意力。 与 Recall 是一对矛盾 ：扩 K 升 Recall 降 Precision。优化：加 Rerank、收紧阈值。工程平衡：大 K（20-50）保 Recall → Rerank 拉 Precision → 最终 3-5 条进 LLM
MRR 平均倒数排名	第一个相关结果的排名倒数，在多个查询上取平均 $\text{MRR} = \frac{1}{N} \times \sum \frac{1}{\text{rank}_i}$ 例：3 个查询的第一个相关结果排在第 1、3、2 位 → $\text{MRR} = \frac{1}{3} \times (\frac{1}{1} + \frac{1}{3} + \frac{1}{2}) = 0.611$	衡量“最相关的结果是否排在前面”，对用户体验影响大（用户只看前几条）。MRR vs Recall：Recall 看有没有找全，MRR 看最好的有没有排前面
NDCG 归一化折损累积增益	考虑排序位置和相关度等级的综合指标。三步：① 相关度分级（如 0-3 分）；② 位置折扣——排前面权重高；③ 与理想排序对比归一化到 0-1 $\text{DCG@K} = \sum (\text{rel}_i / \log_2(i+1))$ $\text{NDCG@K} = \text{DCG@K} / \text{IDCG@K}$	最全面但标注成本高（需人工标相关度等级），适合离线精确对比两个方案的排序质量；日常监控更多用 Recall 和 MRR

10.3 生成侧评估：RAGAS 四维度

生成侧难点：答案是自然语言、没有标准答案，很难用简单公式衡量。RAGAS（Retrieval Augmented Generation Assessment）框架提出基于 LLM 的自动化评估方法，已成为事实标准。

维度	评估方法与公式	意义
Faithfulness 忠实度	① 把答案拆成多个陈述；② LLM 判断每个陈述能否从上下文推导；③ = 可推导陈述数 / 总陈述数。例：上下文说"年假 10 天，满 3 年 15 天"，答案多说了"满 5 年 20 天" → $2/3 = 66.7\%$	衡量是否真正 Grounded——RAG 相比纯 LLM 的核心优势。低说明模型自由发挥；优化：改进 Prompt 约束、用更强模型、减少上下文噪声
Answer Relevancy 答案相关性	① LLM 根据答案反向生成 N 个可能的问题；② 算这 N 个问题与原始问题的语义相似度均值	防"答非所问"。低说明答案偏题（如问年假政策，答成弹性工作制）
Context Relevancy 上下文相关性	LLM 从每个 chunk 提取与问题相关的句子，= 相关句子数 / 总句子数。例：chunk 四句话只有"年假 10 天"相关 → 25%	衡量检索与 Chunking 质量，从生成角度反向评估检索效果。优化：改 Chunking、加 Rerank、减 top-K
Context Recall 上下文召回	需有标注标准答案（ground truth）：LLM 判断标准答案每个陈述能否从上下文推导，= 可推导陈述数 / 标准答案总陈述数	从生成角度评估检索完整性。低说明检索漏了关键信息——生成再好也给不出完整答案。优化：扩 top-K、改进算法、检查索引覆盖

四指标关系：Context Recall + Context Relevancy = 检索质量（从生成角度看）；Faithfulness + Answer Relevancy = 生成质量。**Context Recall 低，其他再高也没用**（关键信息根本没检索到）；Context Relevancy 低会拉低 Faithfulness 和 Answer Relevancy（噪声干扰生成）。

10.4 如何构建评估集

评估集三要素：问题（Query）、标准答案（Ground Truth，可选，用于 Context Recall）、相关文档片段（用于检索侧 Recall/Precision）。

方法	质量/成本	说明
人工构造	质量最高 / 成本最高	挑业务高频问题，人工标注正确答案和相关 chunk；适合核心场景，50–200 条即可建立 baseline
真实日志采样	质量中等 / 成本中等	生产日志采样真实用户问题，人工标注答案质量（好/中/差）；适合持续监控
LLM 生成	质量较低 / 成本最低	LLM 根据知识库生成问题-答案对；适合快速验证，但问题可能过于理想化，不代表真实用户行为

工程实践：先用 LLM 生成快速搭框架 → 再人工构造核心场景高质量评估集（50–100 条）→ 用日志采样持续补充扩大覆盖。评估集要有**代表性**，不是一味追求数量（简单场景 50–100 条可建可信 baseline，复

杂开放域可能需 300–500 条)。

10.5 离线评估 vs 在线评估

- **离线评估**：测试集上跑指标、优化前后对比。优点可控、可重复、成本低；缺点测试集可能不代表真实场景。
- **在线评估**：真实用户使用时收集反馈。显式反馈：点赞/点踩、提交纠错；隐式反馈：是否继续追问、是否复制答案、停留时长。
- **关系**：离线评估指导优化方向（必要条件），在线评估验证真实效果（充分条件），两者结合。

10.6 线上采用率：用户真的用了你的答案吗

离线指标再漂亮，用户不采纳就是白搭。**采用率 (Adoption Rate)** 是 RAG 系统最真实的质量信号。

- **直接采用信号 (强)**：复制率 (复制次数/总回答)、点赞率、引用率。
- **间接信号 (弱但规模大)**：无追问率 (也可能只是放弃，需结合停留时长判断)、停留时长 (3–10 秒可能扫一眼，30 秒以上可能认真阅读)、跳出率。
- **负向信号 (问题指标)**：重复提问率 (换问法再问一遍说明没解决)、点踩率、人工接管率。
- **综合采用率示例**： $(\text{复制次数} + \text{点赞次数} \times 2 - \text{点踩次数} \times 2) / \text{总回答次数}$ 。点赞/点踩权重 $\times 2$ ，因为用户主动点击成本比复制高、信号更强。

踩坑案例：离线 Recall@5 从 70% 优化到 85%、Faithfulness 从 0.75 提到 0.88，上线后采用率反而下降。可能原因：① 过度优化测试集 (分布与真实用户不一致)；② 延迟变长 (Rerank+多轮检索让 P95 从 2 秒涨到 5 秒，用户等不及)；③ 答案变长 (把检索内容都塞进答案，用户看不完)；④ 语气变硬 (Prompt 约束太强，答案变成“根据资料显示...”的八股文)。**离线指标是过程指标 (方向盘)**，**采用率是结果指标 (目的地)**。两个都要看，但用户才是最终裁判。

10.7 评估的延迟与成本代价

- RAGAS 四指标都需调 LLM：Faithfulness 约 2–3 次调用、Answer Relevancy 1–2 次、Context Relevancy 调用次数=chunk 数量、Context Recall 1–2 次。按每次 200ms 算，评估一个答案可能额外 1–2 秒；评估 Token 可能是原问答的 4–8 倍。
- **生产系统通常**：线上不做完整评估 (只收集问题、检索结果、答案、用户反馈) → 每天/每周离线采样 1000–5000 条跑完整 RAGAS → 线上只做轻量监控 (答案长度、检索延迟、反馈率)。
- **分级评估策略**：高风险查询 (5%，涉金额/法律) → 实时 RAGAS 全量 (延迟+2 秒可接受)；中风险 (20%) → 实时 Faithfulness 单项 (+500ms)；低风险 (75%) → 仅记日志离线评估 (无额外延迟)。
- **省钱四招**：采样评估 (每天随机 1%–5% 跑完整评估)、分级评估、只跑当前优化目标相关的 1–2 个指标、用便宜模型做评估判断 (成本降 10 倍、准确率只降 5%)。

10.8 评估驱动优化闭环

1. **建立 baseline**：评估集上跑当前系统记录所有指标，作为对比基准

2. **定位瓶颈**：Recall@5 < 70% → 优先优化检索召回；Recall 高但 Precision 低 → 加 Rerank；Context Relevancy 低 → 优化 Chunking 或减 top-K；Faithfulness 低 → 改进 Prompt 约束；Answer Relevancy 低 → 检查 Query 理解和模型能力
3. **针对性优化**：每次只改一个变量，评估集重跑指标
4. **对比验证**：比 baseline 好 → 采纳；更差或某些场景退化 → 回滚或继续调整
5. **持续迭代**：新问题加入评估集、定期重新评估、监控线上指标

开源工具：检索评估自写脚本或 `ranx`；RAGAS 评估直接用 `ragas` 库（支持多种 LLM 后端）；实验跟踪用 LangSmith 或 Weights & Biases。

面试 Q：怎么快速讲清 RAG 评估？

三层结构：检索侧指标 (Recall/Precision/MRR/NDCG) + 生成侧指标 (RAGAS 四维度) + 评估驱动优化闭环。再举例："我们发现 Recall@5 只有 65%，优化 Embedding 后提升到 82%；再加 Rerank 后 Precision 从 30% 提到 70%；最终答案 Faithfulness 从 0.75 提升到 0.88。"——有框架、有数据、有闭环。

拓展 Q：Recall 和 Precision 哪个更重要？

看业务：答不上代价高（医疗、法律咨询）→ Recall 更重要，宁多召回不漏关键信息；噪声代价高（客服答错被投诉）→ Precision 更重要。通常做法：大 K 保 Recall、Rerank 保 Precision，两头都要。

拓展 Q：RAGAS 依赖 LLM 判断，会不会不准？

可能有偏差。工程实践：在核心评估集上对比 LLM 评估与人工评估的一致性，一致性高（85%+）就信任 LLM 做大规模自动化；一致性低则调整评估 Prompt 或引入人工。

十一、面试回答框架

11.1 面试官怎么问：沿项目一路下挖

面试官通常不会随机抽十个名词让你背，而是从简历上一句"负责企业知识库 RAG 系统"开始，顺着同一条项目链追问：**为什么做** → **链路是什么** → **参数怎么定** → **效果差怎么排** → **收益怎么证明**。链路、选型、故障和指标属于**必答区**；索引算法源码、Embedding 训练目标等底层细节，知道原理和工程影响即可，不要硬装算法专家。**真正拉开差距的不是背到多底层，而是必答区里每一层都有项目证据。**

11.2 RAG 项目面试五个落点

- **业务目标**：知识从哪来，为什么需要 RAG，不用 RAG 会出什么问题
- **完整链路**：离线怎么建库，在线怎么检索、重排、拼接和生成
- **选型取舍**：Chunk、Embedding、向量库、Top-K、混合检索为什么这么选
- **问题诊断**：答案不准时，先区分检索没找到，还是模型没用好
- **指标证据**：用检索、生成、线上业务三层指标证明方案有效

11.3 通用回答五步框架

回答顺序：① **结论（先直接回答）** → ② **项目事实（数据、场景、症状、约束）** → ③ **方案取舍（为什么选 A 不选 B，代价是什么）** → ④ **指标证据（基线和改后分别怎样）** → ⑤ **适用边界（什么情况下这套方案不成立）**。上来先报技术栈（"LangChain、Milvus、BGE、Rerank 全用了"）相当于把工具箱倒在桌上，却没说修的是什么问题。

示例——面试官问"为什么加 Rerank"：不要只说"Rerank 能提高准确率"。可以这样答："我们先看评估集，发现相关 Chunk 多数已进入 Top-20，但真正能回答问题的片段经常排不到前 5，所以问题不在召回覆盖，而在候选排序。我们保留前面的快速召回，再用 Rerank 做精排，只把 Top-N 交给模型。代价是多了一段推理延迟，所以候选数和 Top-N 是根据离线排序指标、端到端答案质量和 P95 延迟一起定的。"——有症状、有判断、有方案、也有代价。

11.4 高频八问与回答要点

问题	踩雷答法	回答框架
Q1 为什么这个项目要用 RAG?	"因为大模型有幻觉"——太泛，幻觉不代表所有场景都要建知识库	业务缺口 → RAG 能力 → 不选其他方案的原因。RAG 解决的是 外部知识接入 ，不是给所有模型问题贴创可贴；动态知识不适合训进参数；固定格式/语气问题先用 Prompt，必要时才微调
Q2 从提问到回答的完整链路?	"文档向量化、相似度检索、大模型回答"——只有三个大词	离线建库 + 在线问答 + 质量控制（生成后还要做引用、无答案拒答、权限和日志监控）

Q3 Chunk 多大？Overlap 多少？为什么？	"Chunk 500、Overlap 100，常见配置"——"常见"不是选型依据，追问"为什么不是300"就断了	文档结构 → 用户问题粒度 → 参数候选 → 评估结果。好回答 不是报出神奇数字，而是说清数字怎么来的 ；Overlap 只保护边界信息，过大会造成重复召回和上下文浪费
Q4 Embedding 和向量数据库怎么选？	答成产品点名大会	两者不是一回事。Embedding 看效果+部署约束（语言/领域匹配、自有评估集上的 Recall@K/MRR/NDCG、维度带来的存储检索成本、部署延迟吞吐合规）；向量库看数据规模+工程能力（数据量并发、索引类型延迟、持久化扩缩容高可用、元数据过滤/增量更新/多租户、运维成本）。收口："先用业务评估集验证召回效果，再结合延迟、成本、合规和运维约束做选择"
Q5 为什么混合检索？为什么还要 Rerank？	"生产环境标配混合检索加 Rerank"	混合检索补召回，Rerank 提排序 。向量擅长语义但精确词（型号/错误码/编号/人名）可能漏；BM25 擅长关键词命中但不理解改写。主动说代价：混合检索多一套链路、Rerank 多推理延迟、Recall 本来低时 Rerank 救不回。工程里没有无条件标配，只有指标和约束下的选择
Q6 RAG 答不准怎么排查？	第一步就换模型或加 Rerank	先看 Top-K 候选里有没有支撑正确答案的 Chunk：没有 → 查解析/索引覆盖/Chunk/Embedding/Query/召回策略；有但排序靠后 → 查融合/Rerank/Top-K/上下文噪声；有且位置合理仍错 → 查 Prompt 拼接/引用约束/上下文冲突/模型能力；知识库本来没有 → 拒答或转人工，不能逼模型编
Q7 你们做了哪些优化？	"Query 改写、混合检索、Rerank、父子块、Context 压缩全做了"——一次改五个变量不知道谁起作用	先建 Baseline，把失败样本按检索侧/生成侧分类，每次只调一个变量，用同一评估集比较改前改后，达到收益门槛才保留否则回滚。只选 1-2 个真正做过的讲透：优化前症状 → 根因定位 → 为什么选这个方案 → 哪些指标变化
Q8 怎么证明 RAG 真的有效？	"效果提升挺明显"	指标分三层： 检索侧 （Recall@K/Precision@K/MRR/NDCG）； 生成侧 （忠实度/答案相关性/引用正确率/无答案正确拒答率）； 线上业务侧 （任务成功率、反馈/转人工率、P95/P99 延迟、超时率、单次请求成本、越权检索等风险指标）。句式："优化前 [指标] 是 [基线]；定位到 [根因] 后做了 [单一改动]；指标变成 [结果]；同时 [延迟/成本] 变化 [真实数据]，所以选择 [上线/灰度/放弃]"。忘了数值就讲统计口径和区间， 不能现场编

11.5 面试里最容易暴露的四个问题

- **只报技术栈，不讲问题：**工具名只能证明“知道”。改法：每个工具后面补一句“它解决了什么问题，为什么适合当前约束”。
- **把参数说成行业标准：**Chunk 大小、Overlap、Top-K、阈值都依赖数据与任务。改法：讲候选范围、对比实验和最终指标。
- **只讲准确率，不讲成本和延迟：**多一次改写、多一路召回都可能让 P95 延迟和成本失控。改法：质量达标后，比较端到端延迟和每个成功任务成本。
- **编指标：**面试官追问评估集规模、标注方式、统计口径时容易穿帮。改法：没有线上规模就如实说是离线评估；没有自动指标就说明人工抽检规则。

11.6 只做过 Demo 怎么答

不要硬说自己做过生产级 RAG。可以明确说明：“这个项目目前做到离线 Demo，没有真实线上流量，所以我验证的是检索和生成质量，没有声称做过高并发生产验证。”然后继续讲：如何构造测试问题和标准证据、对比过哪些方案、失败样本暴露了什么问题、如果上线还要补哪些监控安全和容量工作。**诚实地讲清边界，比虚构一个生产项目更像工程师。**

面试官追问没负责的环节时：直接说明责任边界，再讲你理解的接口和影响。例如：“索引集群由基础设施同学维护，我负责检索策略和评估；我关注索引参数对召回、延迟和内存的影响，但没有改过底层实现。”这比把团队成果全揽到自己身上更可信。

11.7 知识拓展快问快答

- **Top-K 是不是越大越好？** 不是。K 增大提高召回覆盖，但带来更多噪声、Token、延迟和生成干扰。先看 Recall@K 曲线，再结合 Rerank 后的上下文质量和端到端指标选。
- **有了 Rerank 还需要优化 Embedding 吗？** 需要。Rerank 只能重排已召回的候选；相关内容根本没进候选池，它无能为力。先保证召回，再优化排序。
- **RAG 能彻底消除幻觉吗？** 不能。检索可能找错、文档可能过期、模型可能忽略或曲解证据。RAG 降低无依据生成概率，但仍要配合引用、拒答、权限、时效性和人工兜底。
- **向量数据库能不能用普通数据库代替？** 看规模和能力需求。数据量小、数据库支持向量类型时可先用现有数据库降低复杂度；当向量规模、查询吞吐、索引能力、过滤和扩缩容要求上来后再评估独立向量数据库。

十二、重点难点提示与复习建议

12.1 重点难点提示

必掌握的核心概念

- **RAG 存在的四大理由**：幻觉、私有知识、时效性、可追溯性——以及每个问题 RAG 如何对应解决、缓解到什么程度（幻觉只能缓解不能根治）。
- **完整链路两分法**：离线（加载→清洗→切片→向量化→入库+元数据）与在线（Query 处理→检索→Rerank→Context→生成），以及"离线在线必须一致"的硬约束。
- **Embedding vs Rerank**：Bi-Encoder 双塔粗筛 vs Cross-Encoder 精排，是全篇最高频的对比题——建模方式、能否预计算、速度规模、精度四个维度都要能讲。
- **ANN 两大索引**：HNSW（分层图导航，快且准，费内存）与 IVF（聚类分货架，省内存，有边界效应）；"近似"的精度代价（95%+）与速度收益。
- **评估指标体系**：检索侧 Recall/Precision/MRR/NDCG + 生成侧 RAGAS 四维度，以及"大 K 保 Recall、Rerank 保 Precision"的工程平衡。

易混易错点

- 换了 Embedding 模型**必须全量重建索引**（不同模型不同语义空间）。
- Recall@K 本身低时加 Rerank **无意义**——先解决召回再优化排序。
- 长文档/代码场景：**先在 Child 小块上排序，再向父级扩展**——顺序写反会稀释信号、抬高延迟。
- 版本与权限：**先过滤后召回**——先召回再删权限会让受限内容进入缓存/日志。
- Agentic RAG ≠ 多 Agent；混合检索/Rerank（单次查准）与 Agentic RAG（该不该查/查几次/不够怎么办）**不在同一层**。
- 离线指标好但线上采用率差的四种原因：测试集偏差、延迟变长、答案变长、语气变硬。
- FAISS 是索引库不是数据库；向量相似度高不等于语义相关（取决于 Embedding 模型质量）。

12.2 复习建议

1. **按框架自测**：合上资料，沿"为什么 → 链路 → 三大基础设施 → 优化 → 排障 → 评估 → 表达"主线，口头复述每一环。卡壳的环节回到对应章节精读。
2. **对比记忆**：把全文成对出现的概念整理成对比卡片——Embedding vs Rerank、HNSW vs IVF、Recall vs Precision、Parent-Child vs Overlap、传统 RAG vs Advanced vs Agentic、离线评估 vs 在线评估。
3. **背指标公式**：Recall@K、Precision@K、MRR、NDCG、RAGAS 四指标的计算方法，配合文中数值例子（如 $MRR = 0.611$ 的计算过程）手推一遍。
4. **练诊断路径**：给自己出题——"RAG 答不准"，限时口述完整排查流程（top-K 证据分流 → 检索侧/生成侧定位 → 单变量优化 → 指标验证），练到 60 秒内讲清。

5. **模拟追问链：**按第十一章的八问，模拟面试官从"为什么用 RAG"一路追问到"怎么证明有效"，每个回答用"结论→事实→取舍→指标→边界"五步组织，并准备一组自己项目的（哪怕是 Demo 的）真实数据。

资料来源：卡码笔记 RAG 系列文章（《为什么有了大模型还需要 RAG》《RAG 完整链路拆解》《Embedding 是什么》《向量数据库解决了什么问题》《长文档与代码怎么检索》《RAG 优化思路》《Agentic RAG》《RAG 切片策略》《RAG 系统答不准的常见问题》《RAG 怎么评估》《面试官怎么问 RAG》），共 11 篇。本文档为结构化整理，内容忠实原文。